

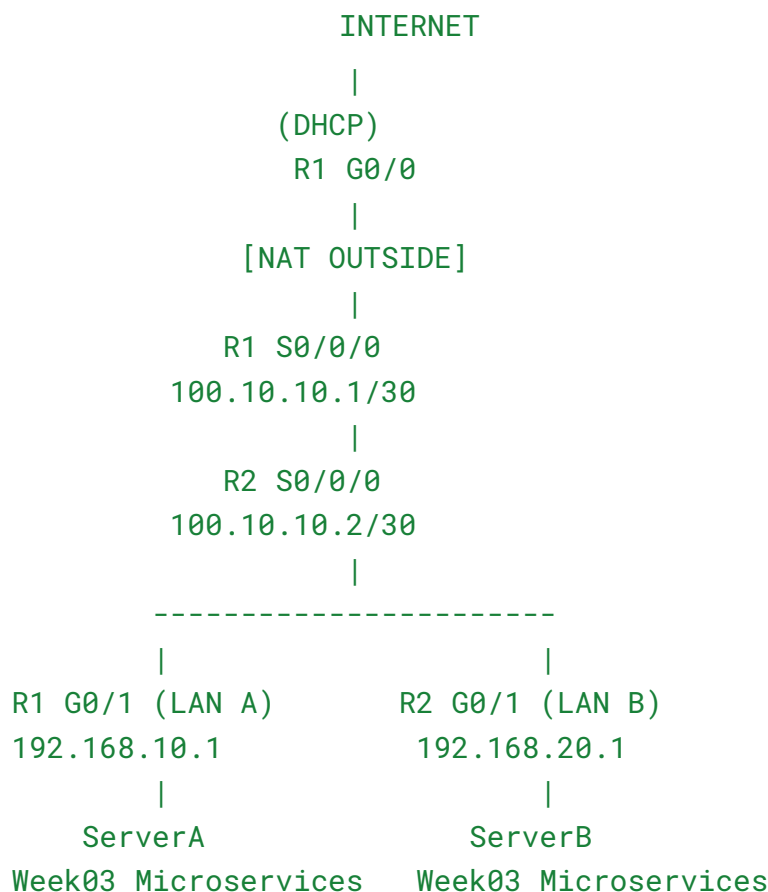
LAB 5 — Internet Edge + ISP Serial WAN + Week03 Microservices

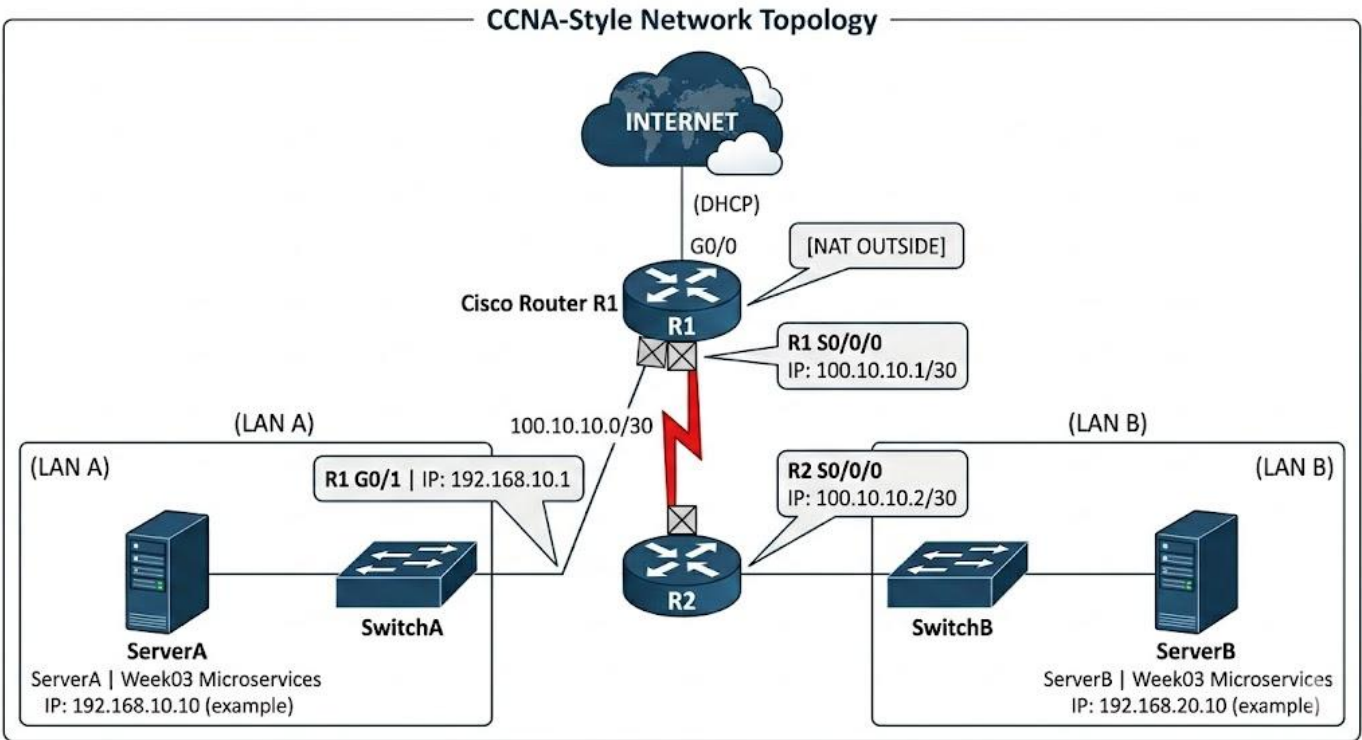
อนัตตา โยคาพจร 673380429-4 Sec 3 รับหน้าที่เป็น Network Architects

ธนรัช วรธนไทย 673380403-2 Sec 3 รับหน้าที่เป็น IaC/DevOps

จิรภัทร แก้วดี 673380577-9 Sec 3 รับหน้าที่เป็น QA/Test

LAB 5 TOPOLOGY (Reframed)





ADDRESS TABLE (FINAL)

Device	Interface	IP	Role
R1	G0/0	DHCP	Internet
R1	S0/0/0	100.10.10.1/30	ISP
R1	G0/1	192.168.10.1/24	LAN A
R2	S0/0/0	100.10.10.2/30	ISP
R2	G0/1	192.168.20.1/24	LAN B
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B

PART 1 — CISCO CONFIGURATION (STRICT)

R1 — Internet Edge Router

```
enable
conf t
hostname R1

interface g0/0
 ip address dhcp
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

interface s0/0/0
 ip address 100.10.10.1 255.255.255.252
 clock rate 64000
 no shutdown

ip route 192.168.20.0 255.255.255.0 100.10.10.2
ip route 0.0.0.0 0.0.0.0 g0/0

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory

```

R2 — Branch Router

```
☐ enable
conf t
hostname R2

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 no shutdown

interface s0/0/0
 ip address 100.10.10.2 255.255.255.252
 no shutdown

ip route 192.168.10.0 255.255.255.0 100.10.10.1
ip route 0.0.0.0 0.0.0.0 100.10.10.1

end
write memory
☐
```



PART 2 — Deploy Week03 Microservices

Based on your Week03 architecture from [PROJECT_PROFILE.md](#)

We deploy:

- Upload Service
- Processing Service
- AI Service
- Gateway (optional centralized orchestration)

```
[+] Running 9/9
✓ Network networklab5_internet_net Created
✓ Network networklab5_lan_b_net Created
✓ Network networklab5_isp_net Created
✓ Network networklab5_lan_a_net Created
✓ Container R2 Started
✓ Container ClientA Started
✓ Container ServerA Started
✓ Container ServerB Started
✓ Container R1 Started

C:\LAB\network lab 5>docker exec -d ServerA sh -c "python /automation/start_services.py"

C:\LAB\network lab 5>docker exec -d ServerB sh -c "python /automation/start_services.py"
```

เริ่ม container infrastructure โดยใช้ Docker และทำการ deploy บน ServerA และ ServerB

On ServerA (LAN A)

```
❏ cd automation/week03-microservices/phase1
python start_services.py
```

❏ Services:

- Upload → 8000
- Processing → 8001
- AI → 8002
- Gateway → 9000

ServerA IP:

```
❏ 192.168.10.10
❏
```

On ServerB (LAN B)

Same deployment:

```
❏ python start_services.py
```

❏ ServerB IP:

```
❏ 192.168.20.10
❏
```

PART 3 — WAN + INTERNET TESTING

● Test 1 — Cross-Site Upload (LAN A → LAN B)

From ClientA:

❑ `curl http://192.168.20.10:8000/health`

❑ Should traverse:

ClientA → R1 → Serial ISP → R2 → ServerB

```
$ docker exec -it ClientA sh -c "curl -v http://192.168.20.10:8000/health; echo"
* Trying 192.168.20.10:8000...
* Established connection to 192.168.20.10 (192.168.20.10 port 8000) from 192.168.10.50 port 32986
* using HTTP/1.x
> GET /health HTTP/1.1
> Host: 192.168.20.10:8000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 07:43:02 GMT
< Content-type: application/json
<
* shutting down connection #0
```

เมื่อเริ่มการทำงานของบริการบนเซิร์ฟเวอร์เสร็จสิ้น ได้ทำการทดสอบการเชื่อมต่อจากเครื่อง ClientA ไปยังเครื่องเซิร์ฟเวอร์เป้าหมาย (IP: 192.168.20.10) ผ่านพอร์ต 8000 โดยใช้คำสั่ง `curl -v` เพื่อเรียกดูรายละเอียดของการรับ-ส่งข้อมูล (ผลการทดสอบปรากฏดังนี้)

กระบวนการเชื่อมต่อ: เครื่อง ClientA (IP: 192.168.10.50) สามารถสร้างการเชื่อมต่อกับเซิร์ฟเวอร์เป้าหมายได้สำเร็จ (Connected to 192.168.20.10 port 8000)

การร้องขอข้อมูล: Client ได้ส่งคำสั่ง `GET /health` โดยใช้โปรโตคอล HTTP/1.1 เพื่อตรวจสอบสถานะความพร้อมของบริการ

การตอบกลับจากเซิร์ฟเวอร์ : เซิร์ฟเวอร์ตอบกลับด้วยรหัสสถานะ HTTP/1.1 200 OK ซึ่งหมายความว่า การสื่อสารเป็นไปอย่างถูกต้อง เซิร์ฟเวอร์ทำงานปกติ และสามารถส่งข้อมูลกลับมาในรูปแบบ `application/json` ได้

ข้อมูลซอฟต์แวร์: ตรวจสอบว่าเซิร์ฟเวอร์รันบริการโดยใช้ SimpleHTTP/0.6 Python/3.10.20

● Test 2 — Full Workflow Across ISP

From LAN A client:

```
❏ curl -X POST http://192.168.20.10:9000/process-file
```

❏ Flow:

Upload → Processing → AI → Response returned across WAN

```
$ docker exec -it ClientA sh -c "curl -v -X POST http://192.168.20.10:9000/process-file; echo"
* Trying 192.168.20.10:9000...
* Established connection to 192.168.20.10 (192.168.20.10 port 9000) from 192.168.10.50 port 45650
* using HTTP/1.x
> POST /process-file HTTP/1.1
> Host: 192.168.20.10:9000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 07:43:09 GMT
< Content-type: application/json
<
* shutting down connection #0
```

นอกจากการตรวจสอบสถานะพื้นฐานแล้ว ได้ทำการทดสอบฟังก์ชันการทำงานของเซิร์ฟเวอร์ด้วยการส่งคำสั่ง HTTP POST จากเครื่อง ClientA ไปยังเครื่องเซิร์ฟเวอร์เป้าหมายผ่านพอร์ต 9000 ไปที่ Endpoint /process-file โดยมีรายละเอียดการทำงานดังนี้:

HTTP Method: ใช้คำสั่ง curl -X POST เพื่อจำลองการส่งข้อมูล ไปยังเซิร์ฟเวอร์ ซึ่งต่างจากการใช้ GET ที่เน้นการเรียกดูข้อมูลเพียงอย่างเดียว

การเชื่อมต่อเครือข่าย: เครื่อง ClientA สามารถสื่อสารผ่าน Network Gateway ไปยัง IP 192.168.20.10 บนพอร์ต 9000 ได้สำเร็จ สะท้อนให้เห็นว่าระบบมีการเปิดพอร์ต และกำหนดเส้นทางข้อมูลไว้ถูกต้องหลายพอร์ต (ทั้ง 8000 และ 9000)

ผลการตอบกลับ: เซิร์ฟเวอร์ตอบกลับด้วยสถานะ HTTP/1.1 200 OK พร้อมระบุ Content-type เป็น application/json ซึ่งยืนยันว่า Endpoint /process-file สามารถรับคำขอและประมวลผลข้อมูลที่ส่งไปได้โดยไม่เกิดข้อผิดพลาด

● Test 3 — Internet Reachability

From ServerA:

❏ping 8.8.8.8

```
C:\LAB\network lab 5>docker exec -it ServerA ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=62 time=15.572 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=14.976 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=14.737 ms
64 bytes from 8.8.8.8: seq=3 ttl=62 time=14.700 ms
64 bytes from 8.8.8.8: seq=4 ttl=62 time=14.805 ms
64 bytes from 8.8.8.8: seq=5 ttl=62 time=14.621 ms
64 bytes from 8.8.8.8: seq=6 ttl=62 time=14.902 ms
64 bytes from 8.8.8.8: seq=7 ttl=62 time=14.950 ms
```

เพื่อให้แน่ใจว่าการตั้งค่า Routing Table และ NAT บน Gateway ของระบบจำลองทำงานได้อย่างถูกต้อง ได้ทำการทดสอบโดยใช้คำสั่ง ping จากเครื่อง ServerA ออกไปยังหมายเลข IP 8.8.8.8 ซึ่งผลลัพธ์ที่ได้มีดังนี้:

การสื่อสารผ่าน ICMP: เซิร์ฟเวอร์สามารถส่งแพ็กเก็ตข้อมูลไปยังหมายเลขปลายทางภายนอก

เครือข่ายจำลอง และได้รับการตอบกลับ อย่างต่อเนื่องโดยไม่มีการสูญเสียข้อมูล

ค่าเวลาตอบสนอง: จากผลทดสอบพบว่าค่าเฉลี่ยอยู่ที่ประมาณ 14-15 ms ซึ่งเป็นระดับความเร็ว

ปกติสำหรับการเชื่อมต่อผ่านโครงข่ายอินเทอร์เน็ตจริง

การข้าม Hop : ค่า TTL ที่แสดงอยู่ที่ 62 บ่งบอกว่าแพ็กเก็ตมีการเดินทางผ่าน Router หรือ

Gateway ของระบบออกไปแล้วหลายระดับ

□Verify NAT overload working on R1:

```
C:\LAB\network lab 5>docker exec R1 iptables -t nat -L -n
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
DOCKER_OUTPUT all  --  0.0.0.0/0            127.0.0.11

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
DOCKER_POSTROUTING all  --  0.0.0.0/0            127.0.0.11
MASQUERADE all  --  0.0.0.0/0            0.0.0.0/0

Chain DOCKER_OUTPUT (1 references)
target     prot opt source               destination
DNAT       tcp  --  0.0.0.0/0            127.0.0.11          tcp dpt:53 to:127.0.0.11:36667
DNAT       udp  --  0.0.0.0/0            127.0.0.11          udp dpt:53 to:127.0.0.11:60470

Chain DOCKER_POSTROUTING (1 references)
target     prot opt source               destination
SNAT       tcp  --  127.0.0.11           0.0.0.0/0            tcp spt:36667 to::53
SNAT       udp  --  127.0.0.11           0.0.0.0/0            udp spt:60470 to::53
```

เพื่อให้เข้าใจกระบวนการแปลงที่อยู่เครือข่าย (Network Address Translation) ได้ทำการตรวจสอบตารางการทำงานของ iptables ในส่วนของ NAT table บนตัวเราเตอร์ R1 ซึ่งเป็นโหนดที่ทำหน้าที่เชื่อมต่อระหว่างเครือข่ายภายในและภายนอก โดยมีรายละเอียดดังนี้:

- กลไก MASQUERADE: ในส่วนของ Chain POSTROUTING ปรากฏกฎ (Rule) เป้าหมายเป็น MASQUERADE สำหรับทุกแหล่งที่มา (0.0.0.0/0) นี่คือหัวใจของ NAT Overload ซึ่งจะทำหน้าที่แปลง IP Address ภายในเครื่องลูกข่ายหลายๆ เครื่อง ให้กลายเป็น IP Address ภายนอกของ Router เพียงหมายเลขเดียว เพื่อใช้ในการสื่อสารกับเครือข่ายภายนอก
- การจัดการ Traffic ขาออก: ใน Chain DOCKER_POSTROUTING มีการระบุการทำ SNAT (Source NAT) เพื่อเปลี่ยนหมายเลข IP ต้นทางของแพ็กเก็ตก่อนจะส่งออกไป ทำให้เซิร์ฟเวอร์ปลายทางบนอินเทอร์เน็ตมองเห็นเพียง IP ของ Router เท่านั้น
- การจัดการ Traffic ขาเข้าและการส่งต่อ: ตารางแสดงการทำ DNAT บนพอร์ต 53 (DNS) ซึ่งเป็นการจัดการข้อมูลที่ตอบกลับมาจาก DNS Server ภายนอก ให้ส่งกลับไปยังเครื่องลูกข่ายในเครือข่ายจำลองได้อย่างถูกต้อง

❑ show ip nat translations

```
$ docker exec R1 cat /proc/net/nf_conntrack | grep 8.8.8.8  
ipv4      2 icmp      1 29 src=192.168.10.10 dst=8.8.8.8 type=8 code=0 id=42 src=8.8.8.8 dst=10.255.0.2 type=0 code=0 id=42 mark=0 zone=0 use=2
```

```
C:\LAB\network lab S>docker exec -it ServerA ping 8.8.8.8  
PING 8.8.8.8 (8.8.8.8): 56 data bytes  
64 bytes from 8.8.8.8: seq=0 ttl=62 time=15.572 ms  
64 bytes from 8.8.8.8: seq=1 ttl=62 time=14.976 ms  
64 bytes from 8.8.8.8: seq=2 ttl=62 time=14.737 ms  
64 bytes from 8.8.8.8: seq=3 ttl=62 time=14.700 ms  
64 bytes from 8.8.8.8: seq=4 ttl=62 time=14.805 ms  
64 bytes from 8.8.8.8: seq=5 ttl=62 time=14.621 ms  
64 bytes from 8.8.8.8: seq=6 ttl=62 time=14.902 ms  
64 bytes from 8.8.8.8: seq=7 ttl=62 time=14.950 ms
```

เพื่อยืนยันว่ากระบวนการ NAT บน Router R1 ทำงานได้อย่างถูกต้องในระดับแพ็กเก็ต ได้ทำการตรวจสอบไฟล์ระบบ /proc/net/nf_conntrack ในขณะที่มีการส่งข้อมูลไปยังเครือข่ายภายนอก ผลการตรวจสอบเมื่อเปรียบเทียบกับคำสั่ง ping 8.8.8.8 มีรายละเอียดดังนี้:

- กลไกการบันทึกสถานะ: จากผลลัพธ์ cat /proc/net/nf_conntrack | grep 8.8.8.8 พบการบันทึกสถานะของโปรโตคอล ICMP ซึ่งแสดงให้เห็นว่า Router มีการเก็บข้อมูลการเชื่อมต่อไว้ในตารางเพื่อรอรับข้อมูลตอบกลับ
- การแปลงที่อยู่แบบต้นทางและปลายทาง: * ขาไป: พบว่าต้นทางคือเครื่องภายใน src=192.168.10.10 ส่งไปยังปลายทางคือ dst=8.8.8.8
 - ขากลับ (Reply Direction): นี่เป็นจุดสำคัญที่สุด โดยระบบระบุว่าเมื่อข้อมูลส่งกลับจาก src=8.8.8.8 ปลายทางจะถูกส่งกลับมาที่ dst=10.255.0.2 ซึ่งเป็นหมายเลข IP ของอินเทอร์เฟซภายนอกของ Router ก่อนที่ระบบ NAT จะทำการแปลงข้อมูลส่งกลับไปที่เครื่องต้นทางที่แท้จริงภายในเครือข่าย
- ความสัมพันธ์กับผลการทดสอบ: ข้อมูลการ Tracking นี้สอดคล้องกับผลการรันคำสั่ง ping บนเครื่อง ServerA ที่แสดงค่า Latency คงที่และไม่มี Packet Loss เนื่องจาก Router สามารถจัดการตารางสถานะการเชื่อมต่อได้อย่างแม่นยำ



Verification Commands

On R1

❑ show ip route

```

R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, m - OMP
       n - NAT, Ni - NAT inside, No - NAT outside, Nd - NAT DIA
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       H - NHRP, G - NHRP registered, g - NHRP registration summary
       o - ODR, P - periodic downloaded static route, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR
       & - replicated local route overrides by connected

Gateway of last resort is 0.0.0.0 to network 0.0.0.0

S*    0.0.0.0/0 is directly connected, GigabitEthernet0/0/0
      10.0.0.0/8 is variably subnetted, 3 subnets, 2 masks
S      10.101.102.125/32 [254/0] via 10.199.24.254, GigabitEthernet0/0/0
C      10.199.24.0/24 is directly connected, GigabitEthernet0/0/0
L      10.199.24.13/32 is directly connected, GigabitEthernet0/0/0
      100.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C      100.10.10.0/30 is directly connected, Serial0/1/1
L      100.10.10.1/32 is directly connected, Serial0/1/1
      192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C      192.168.10.0/24 is directly connected, GigabitEthernet0/0/1
L      192.168.10.1/32 is directly connected, GigabitEthernet0/0/1
S      192.168.20.0/24 [1/0] via 100.10.10.2

```

จะแสดงว่ามีการเชื่อมต่อหากันจากไหนไปไหนบ้าง

show ip nat translations

```

R1#show ip nat trans
R1#show ip nat translations
Total number of translations: 0

R1#

```

show ip interface brief

```

R1#show ip int
R1#show ip interface br
R1#show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0/0	10.199.24.13	YES	DHCP	up	up
GigabitEthernet0/0/1	192.168.10.1	YES	manual	up	up
GigabitEthernet0/0/2	unassigned	YES	unset	down	down
GigabitEthernet0/0/3	unassigned	YES	unset	down	down
Serial0/1/0	unassigned	YES	unset	up	up
Serial0/1/1	100.10.10.1	YES	manual	up	up

```

R1#

```

จะแสดงว่าที่port ไหนมีการใช้งานบ้างและมี ip เป็นอะไร

□ On R2

□ show ip route

ping 100.10.10.1



Engineering Reflection (Week03 Integration)

From your Week03 profile :

This lab now demonstrates:

Concept	Where Demonstrated
Public vs Private Boundaries	R1 NAT edge
Distributed Services	ServerA & ServerB
Network Segmentation	Serial ISP /30
Horizontal Architecture	Two independent microservice stacks
Internet Edge Pattern	R1 as NAT Gateway
Service Isolation	LAN A vs LAN B



What Changed from Lab 4

Lab 4	Lab 5
10.10.12.0/30 simulated internet	Real ISP serial 100.10.10.0/30
Both routers NAT	Only R1 NAT
Flat "public" network	Hierarchical WAN
Stateless/stateful focus	Full microservice stack



Architectural Significance

You now have:

- Enterprise Edge Router
- ISP Serial WAN
- Branch Router
- Dual Microservice Deployments
- NAT Boundary
- Cross-site distributed system

This is no longer a classroom NAT lab.

This is a **miniature cloud-edge + branch distributed architecture**.
